**Room 1_Web Application Basics**

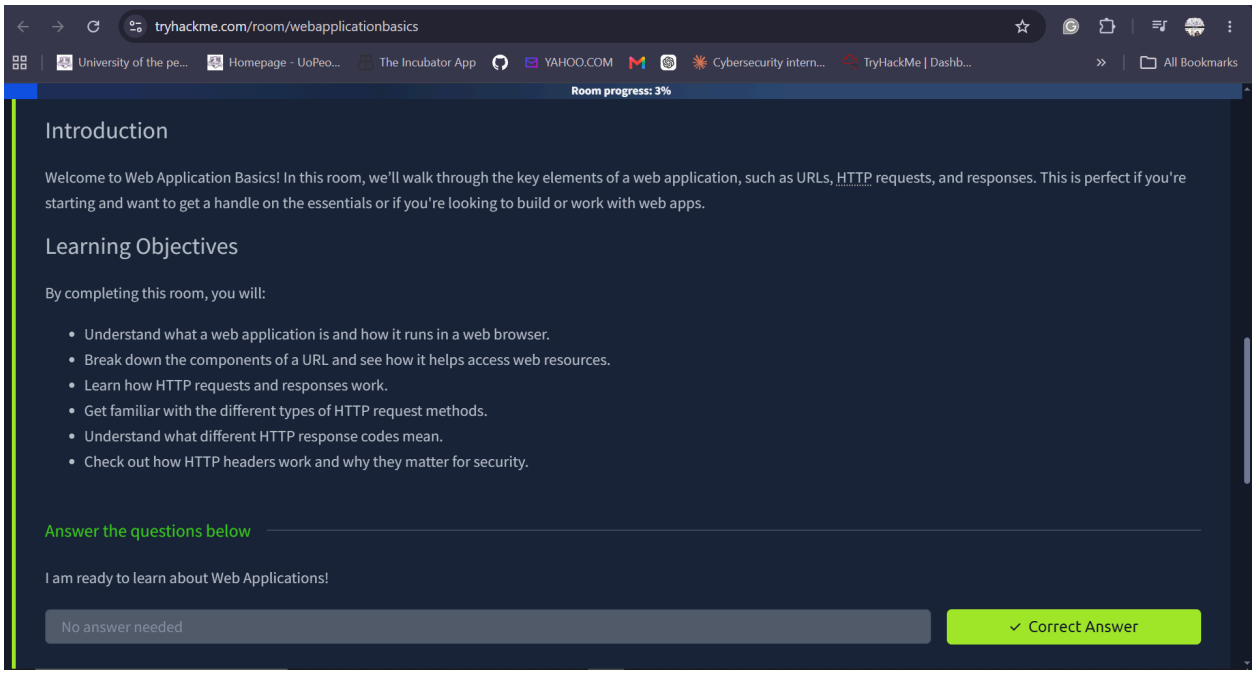
**Platform:** TryHackMe
**Objective:** Understand the components of a web application, how URLs and HTTP messages work, and why headers matter for security.

**Task 1: Introduction**

**Notes:**

This room introduces the key parts of a web application: URLs, HTTP requests and responses, methods, status codes, and headers. By the end, I should understand how a web app runs in a browser and why HTTP headers matter for security.

**Screenshot:**




**Task 2: Web Application Overview**

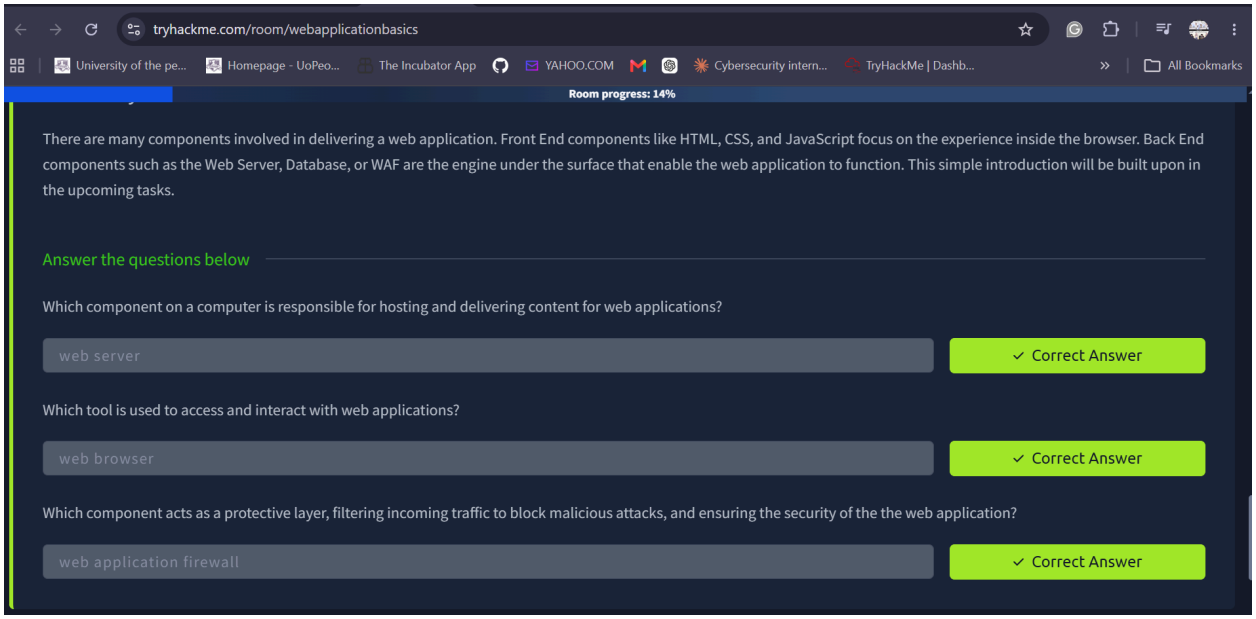
**Notes:**

A web application has a **front end** (what the user sees) and a **back end** (what runs behind the scenes).

- **Front end:** HTML defines the content, CSS controls the appearance, and JavaScript adds behaviour and decision-making in the browser.
- **Back end:** the database (stores and retrieves information), the infrastructure (web servers, application servers, networking), and the optional **WAF**, which filters dangerous requests before they reach the web server.

The room compares this to a planet: the front end is the surface you can see, and the back end is everything underneath that makes it work.

**Screenshot:**



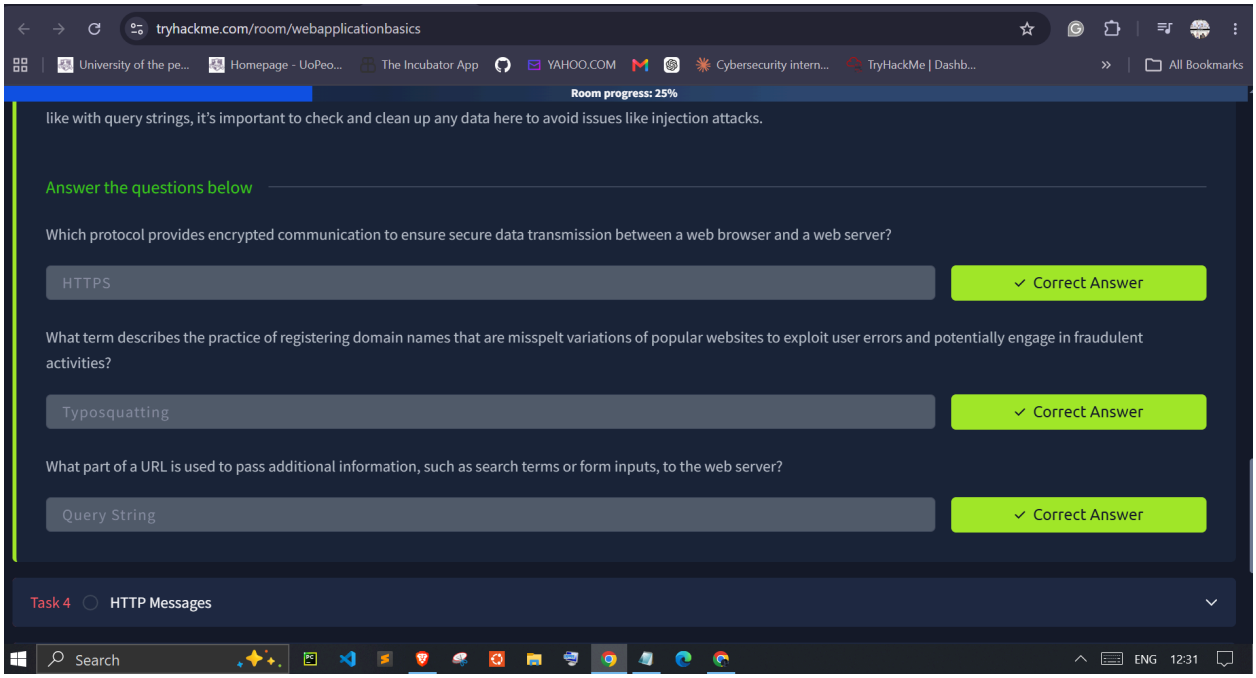

**Task 3: Uniform Resource Locator (URL)**

**Notes:**

A URL is made of several parts: **scheme** (HTTP or HTTPS), **user** (rare and unsafe), **host/domain**, **port** (80 for HTTP, 443 for HTTPS), **path**, **query string** (after ?) and **fragment** (after #).

Security points: HTTPS encrypts traffic. **Typosquatting** uses look-alike domains for phishing. Query strings and fragments can be changed by users, so servers must validate them to prevent injection attacks.

**Screenshot:**

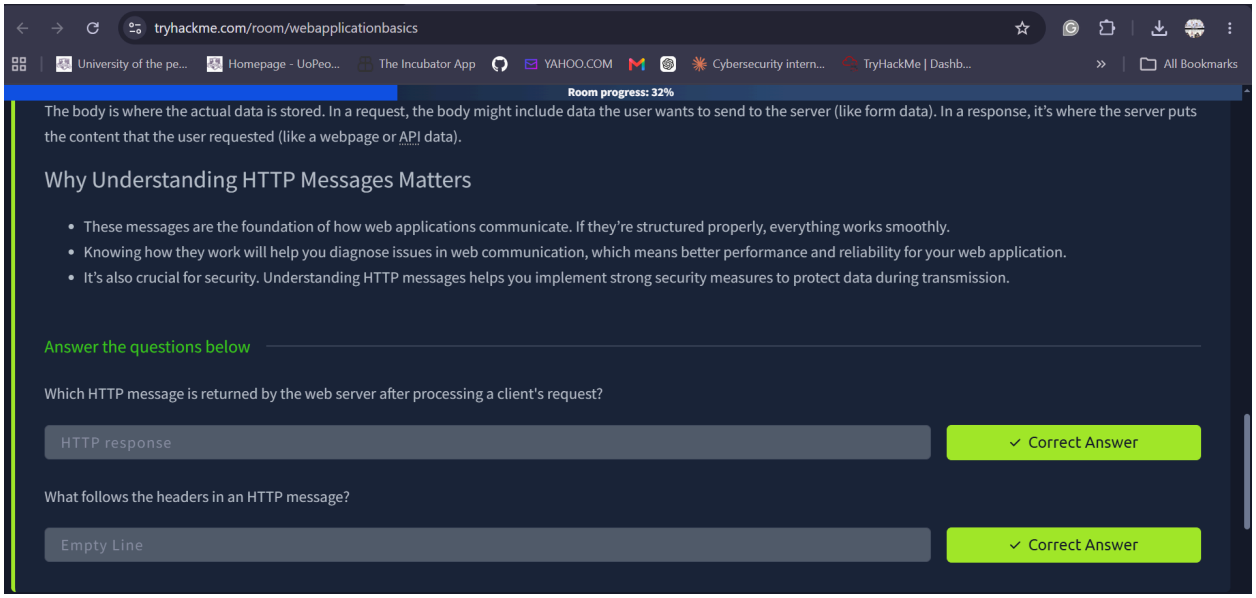



## Task 4: HTTP Messages

## Notes:

HTTP messages are how the client and server communicate. **Requests** go from client to server, and **responses** go back. Both have the same structure: a **start line**, **headers** (key-value pairs), an **empty line** that separates headers from the body, and the **body** that holds the data. The empty line is important because without it the message can be misread.

## Screenshot:



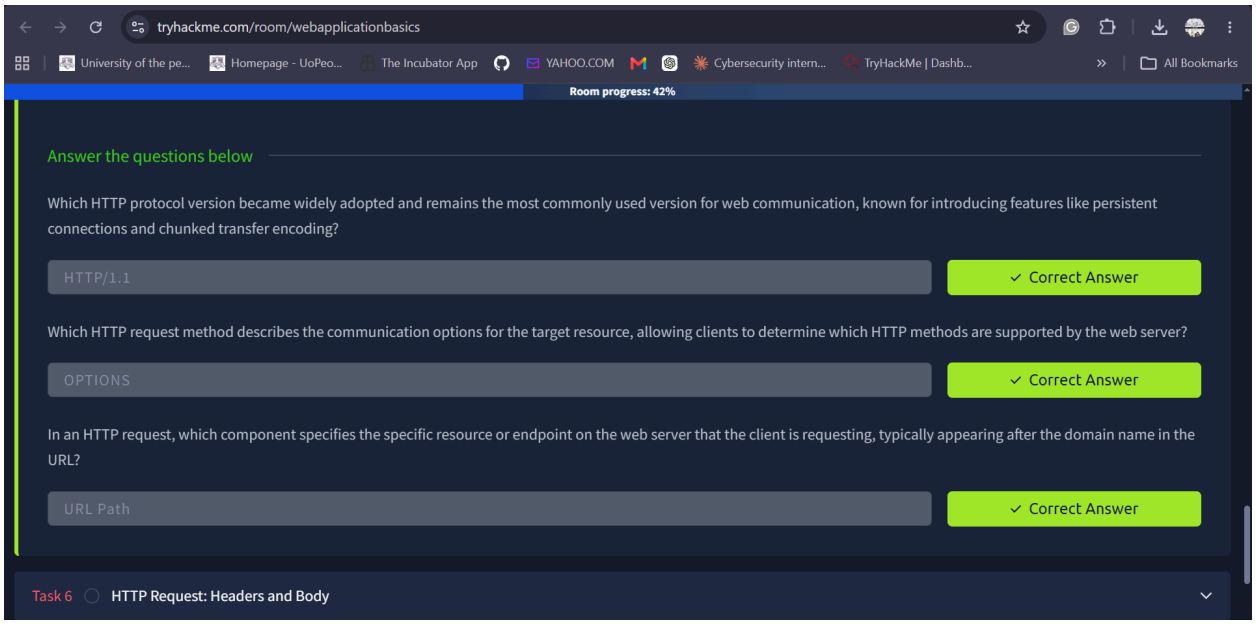

## Task 5: HTTP Request: Request Line

**Notes:**

The request line has three parts: **method**, **URL path** and **HTTP version**.

- **Methods:** GET (fetch), POST (send data), PUT (replace or update), DELETE (remove), PATCH (partial update), HEAD (headers only), OPTIONS (list allowed methods), TRACE (debugging, often disabled) and CONNECT (creates tunnels for HTTPS).
- **Security reminders:** don't put sensitive data in GET requests, validate POST input, and check authorization for PUT and DELETE.
- **HTTP versions:** 0.9 (1991), 1.0 (1996), 1.1 (1997, still the most common), 2 (2015, multiplexing) and 3 (2022, uses QUIC).
- Attackers often manipulate the URL path, so paths must be validated and sanitised.

**Screenshot:**



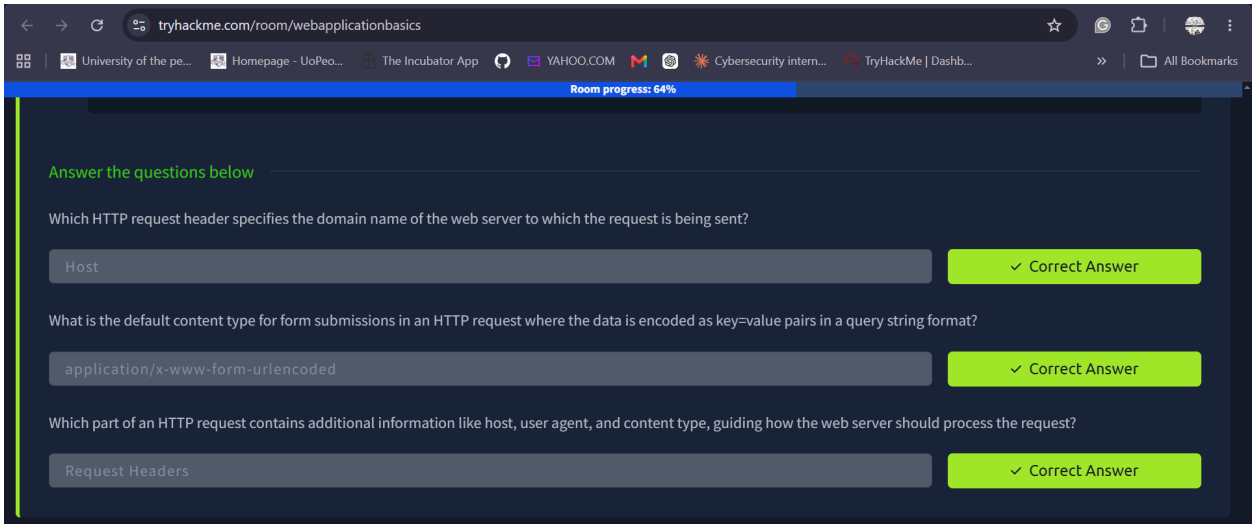

## Task 6: HTTP Request: Headers and Body

**Notes:**

Request headers give the server extra information: Host, User-Agent, Referer, Cookie and Content-Type.
Data sent in the body can use different formats:

- **URL encoded** (application/x-www-form-urlencoded): key=value&key2=value2
- **Form data** (multipart/form-data): used for file uploads
- **JSON** (application/json): {"name": "value"}
- **XML** (application/xml): data inside tags

**Screenshot:**

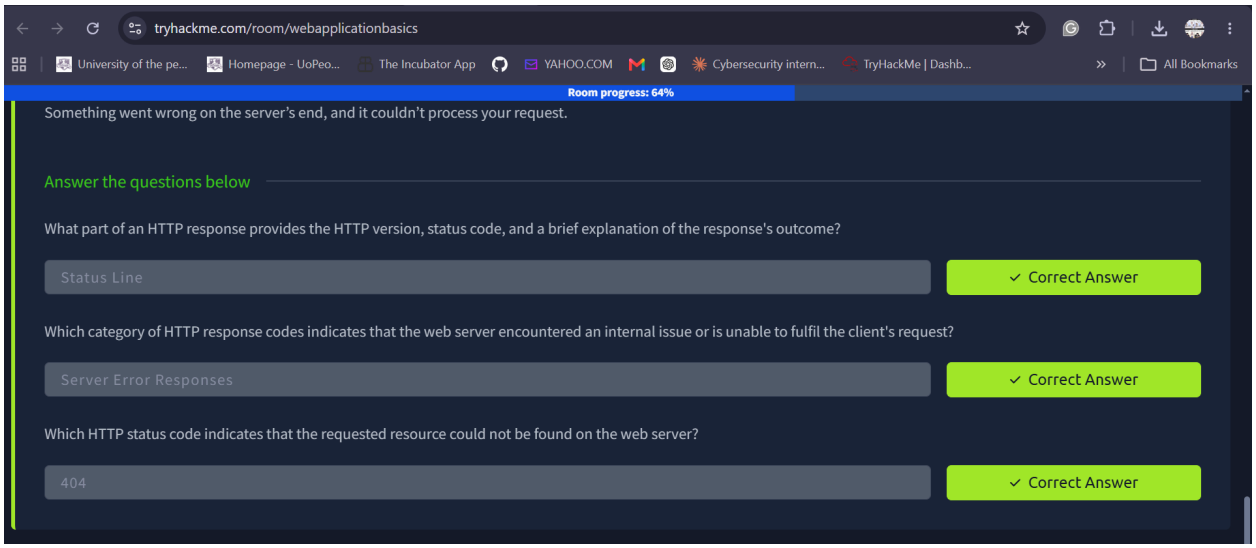



## Task 7: HTTP Response: Status Line and Status Codes

**Notes:**

The first line of a response is the **status line**: version, status code and reason phrase. Codes are grouped as 1xx informational, 2xx success, 3xx redirection, 4xx client error and 5xx server error. Common ones are 200 OK, 301 Moved Permanently, 404 Not Found and 500 Internal Server Error.

**Screenshot:**



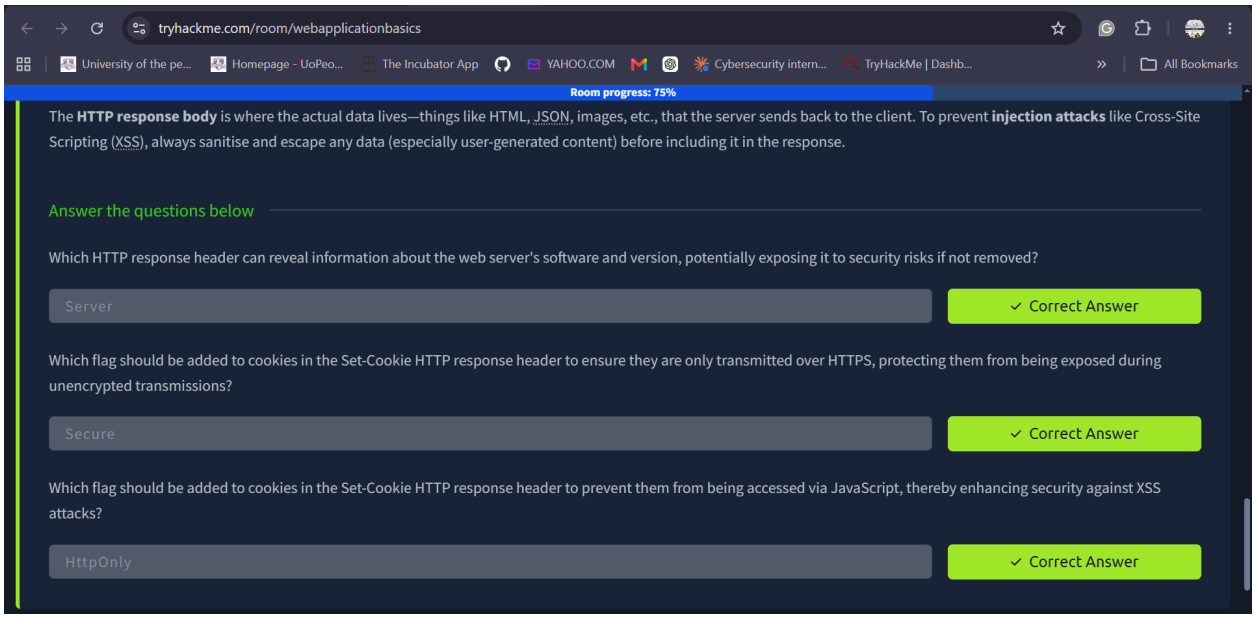
## Task 8: HTTP Response: Headers and Body

**Notes:**

Important response headers include Date, Content-Type, Server, Set-Cookie, Cache-Control and Location.

- The **Server** header can reveal software and version, which helps attackers, so it's often hidden.
- Cookies should use **HttpOnly** (JavaScript can't read them) and **Secure** (HTTPS only).
- If users can influence the **Location** header, it can lead to **open redirect** vulnerabilities.
- Data in the response body must be sanitised and escaped to prevent **XSS**.

**Screenshot:**



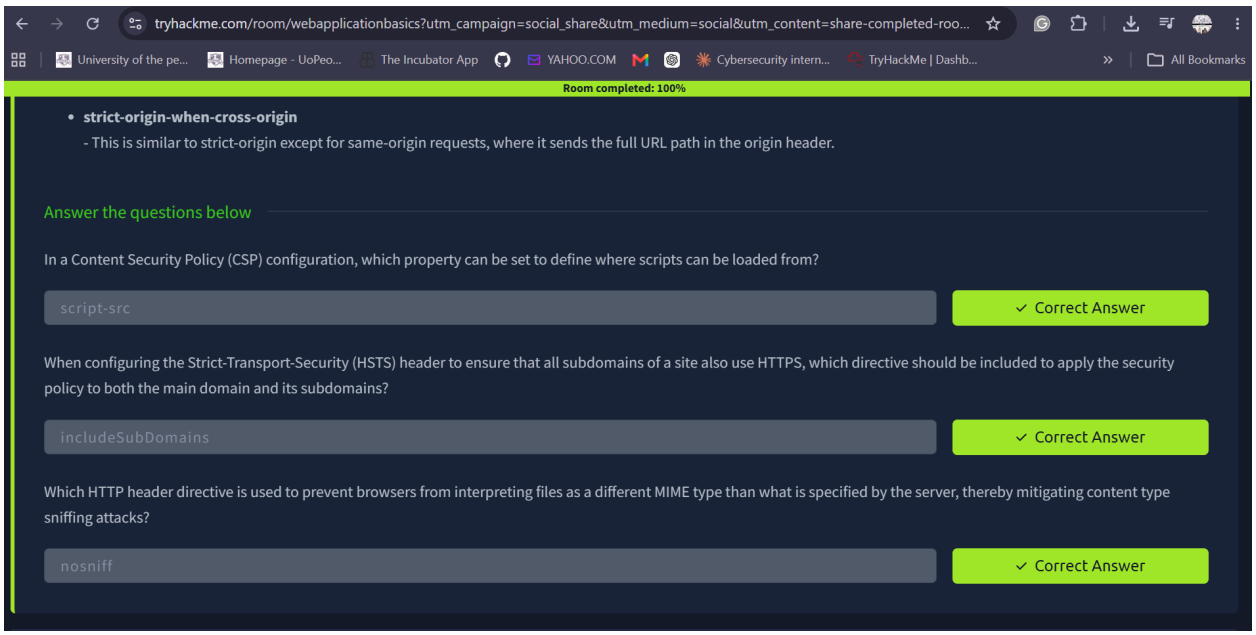

## Task 9: Security Headers

**Notes:**

- **CSP** controls where content can load from, using directives like default-src, script-src and style-src. It helps stop XSS.
- **HSTS** forces the browser to use HTTPS. max-age sets the duration, includeSubDomains applies it to subdomains, and preload lets browsers enforce it before the first visit.

- **X-Content-Type-Options: nosniff** stops the browser from guessing the file type.
- **Referrer-Policy** controls how much referrer information is shared. Values include no-referrer, same-origin and strict-origin.

**Screenshot:**




**Task 10: Hands-on Practice (HTTP Request Emulator)**

This is the practical part, so it's your strongest evidence. Include the request and response for each step.

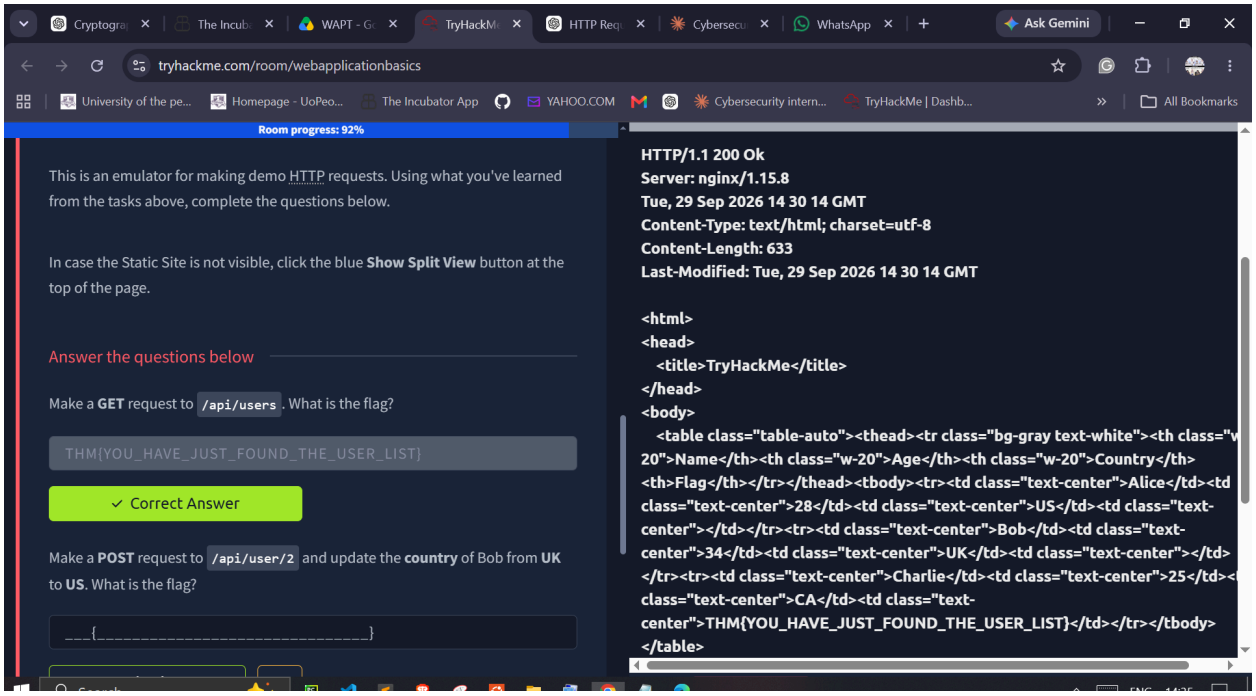
| Step | Request | What it does | Result |
|---|---|---|---|
| 1 | GET /api/users | Retrieves the list of users | 200 OK, flag found in Charlie's row |
| 2 | POST /api/user/2 with country=US | Updates Bob's country from UK to US | 200 OK, "User #2 successfully updated", flag returned |
| 3 | DELETE /api/user/1 | Deletes user 1 | 200 OK, flag returned |

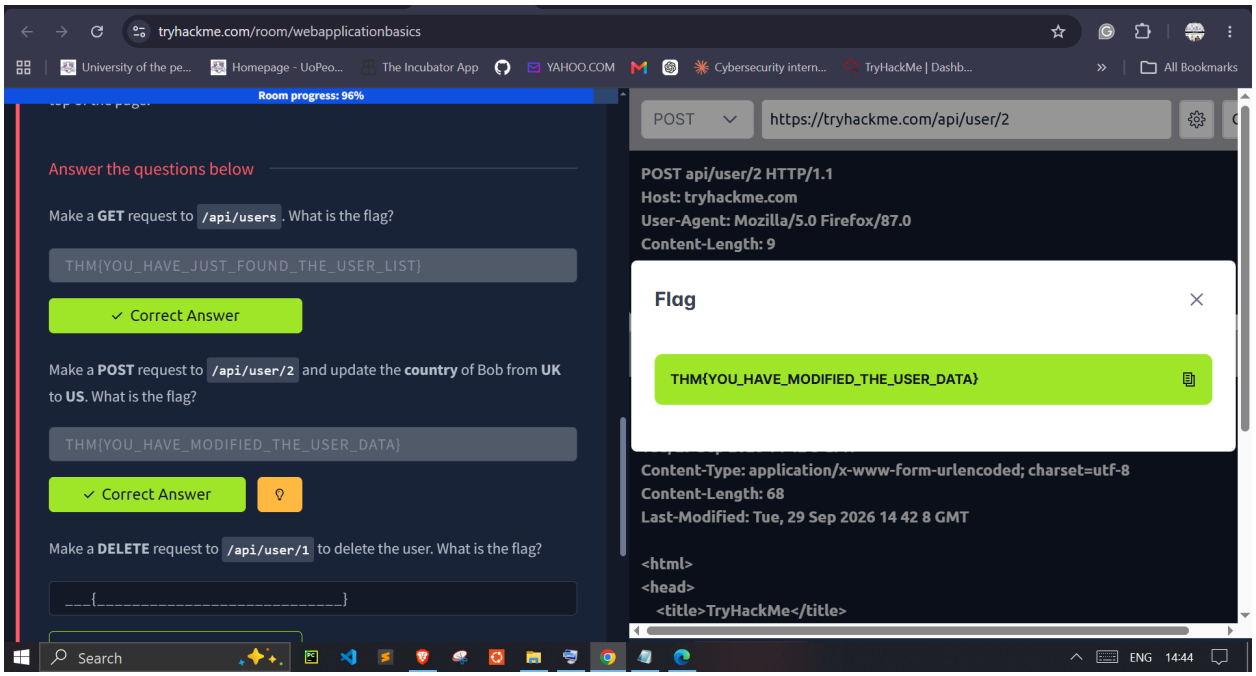
**Notes:**

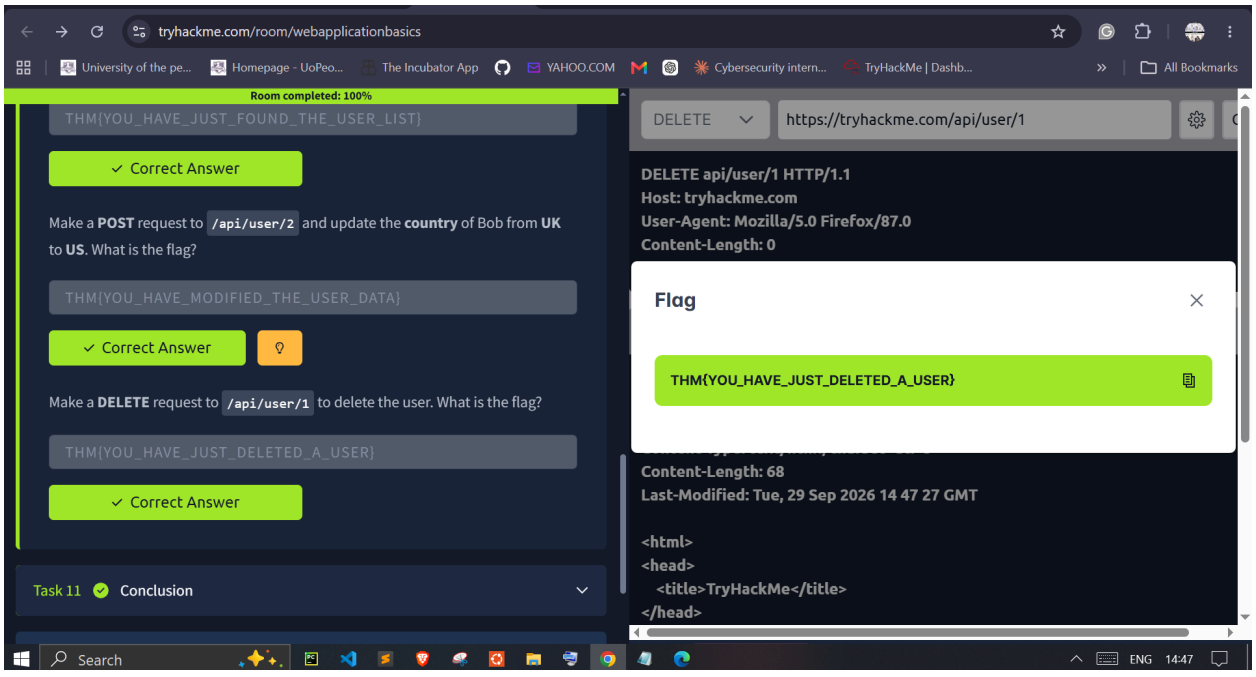
I used the built-in HTTP emulator to send three requests. GET retrieved data, POST sent data in the body to update a record, and DELETE removed a record. I noticed that the response headers included Server: nginx/1.15.8, which reveals the software and version.

**Problem-solving:** on the POST request, my first attempt used Country=US (capital C), and the response didn't include a flag. I corrected the parameter name to lowercase country, which shows parameter names can matter and that a 200 response doesn't always mean the request worked as intended.

**Screenshots:**






## Task 11: Conclusion

**Notes:**

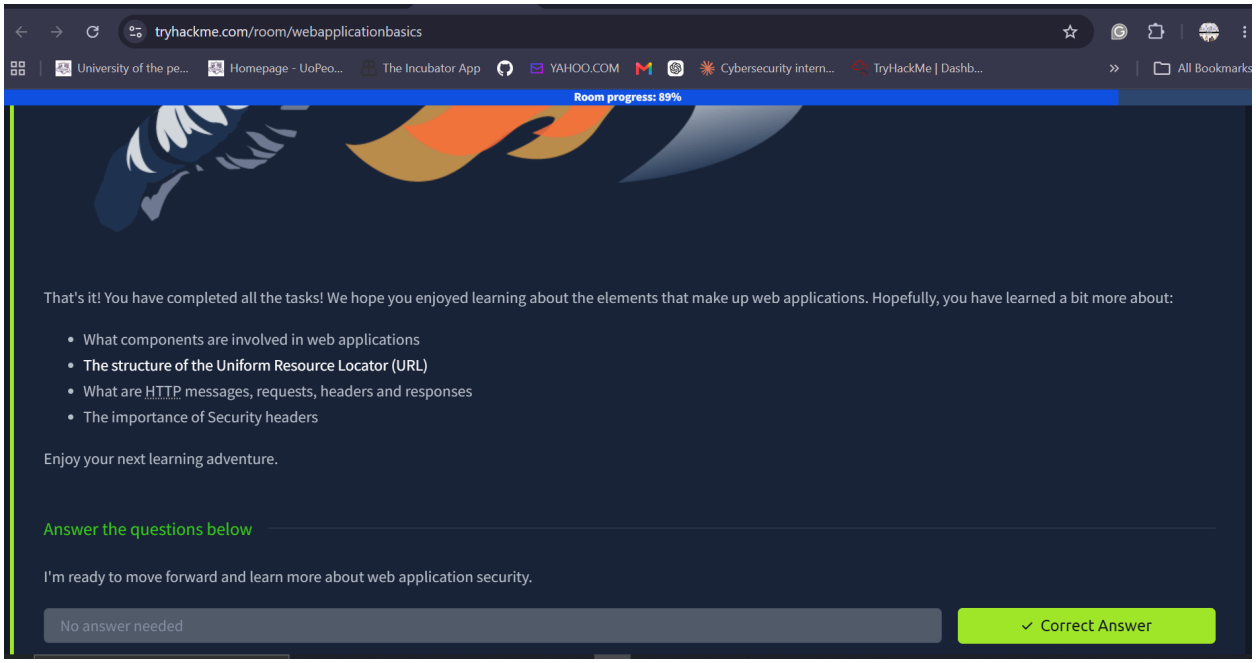
I learned the components of a web application, the structure of a URL, how HTTP requests and responses work, and the role of security headers.

**Security observations to include**

- The /api/users endpoint returned user data without any login, which is an information disclosure and access control concern.
- The POST and DELETE requests changed data with no authentication check. Real applications must verify who is allowed to modify or delete records.
- The Server header exposed the software and version. It should be hidden in production.

**Final "What I learned" paragraph**

This room showed me how a browser and server communicate using HTTP. I learned to read the structure of requests and responses, understand methods and status codes, and recognise security headers. The hands-on task showed me that APIs use GET, POST, and DELETE to read, update, and remove data, and that these actions need proper authorization checks. This gives me a foundation for using Burp Suite to intercept and modify requests later.



**Completion link:**

https://tryhackme.com/room/webapplicationbasics?utm_campaign=social_share&utm_medium=social&utm_content=share-completed-room&utm_source=copy&sharerId=6a298268d8b6b164348fab72

**Room 2_HTTP in Detail**

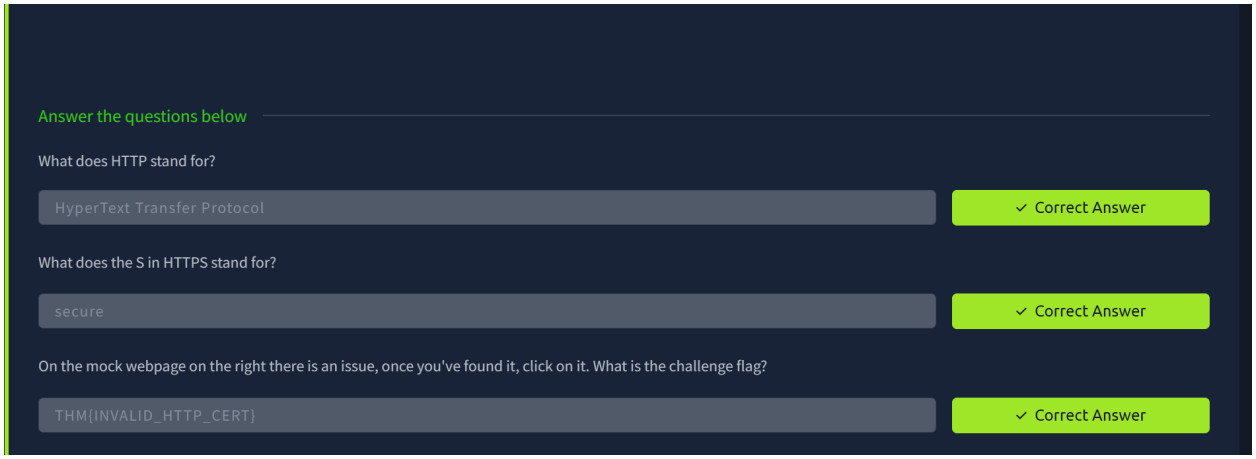
**Platform:** TryHackMe
**Objective:** Understand how HTTP and HTTPS work, including URLs, requests and responses, methods, status codes, headers and cookies.

**Task 1: What is HTTP(S)?**

**Notes:**

> HTTP is the set of rules browsers use to communicate with web servers and transfer web content like HTML, images, and videos. HTTPS is the secure version. It encrypts the data so nobody can read it in transit, and it gives assurance that I'm talking to the real server and not something impersonating it.

**Screenshot:**



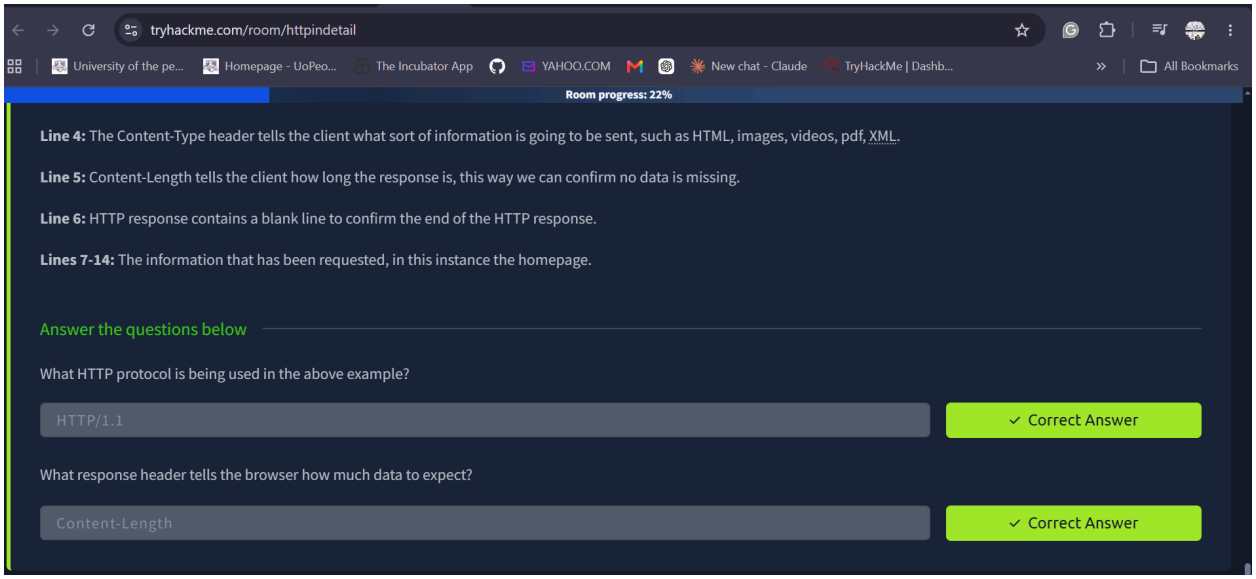

**Task 2: Requests and Responses**

**Notes:**

A URL tells the browser how and where to access a resource. Its parts are the **scheme**, **user**, **host**, **port**, **path**, **query string** and **fragment**. For example, in http://user:password@tryhackme.com:80/view-room?id=1#task3, the query string is ?id=1 and the fragment is #task3.

A request can be as small as one line (GET / HTTP/1.1), but browsers add headers like Host, User-Agent and Referer. A request ends with a blank line. A response starts with the status line (HTTP/1.1 200 OK), followed by headers such as Server, Date, Content-Type and Content-Length, then a blank line and the body (the HTML).

**Screenshot:**



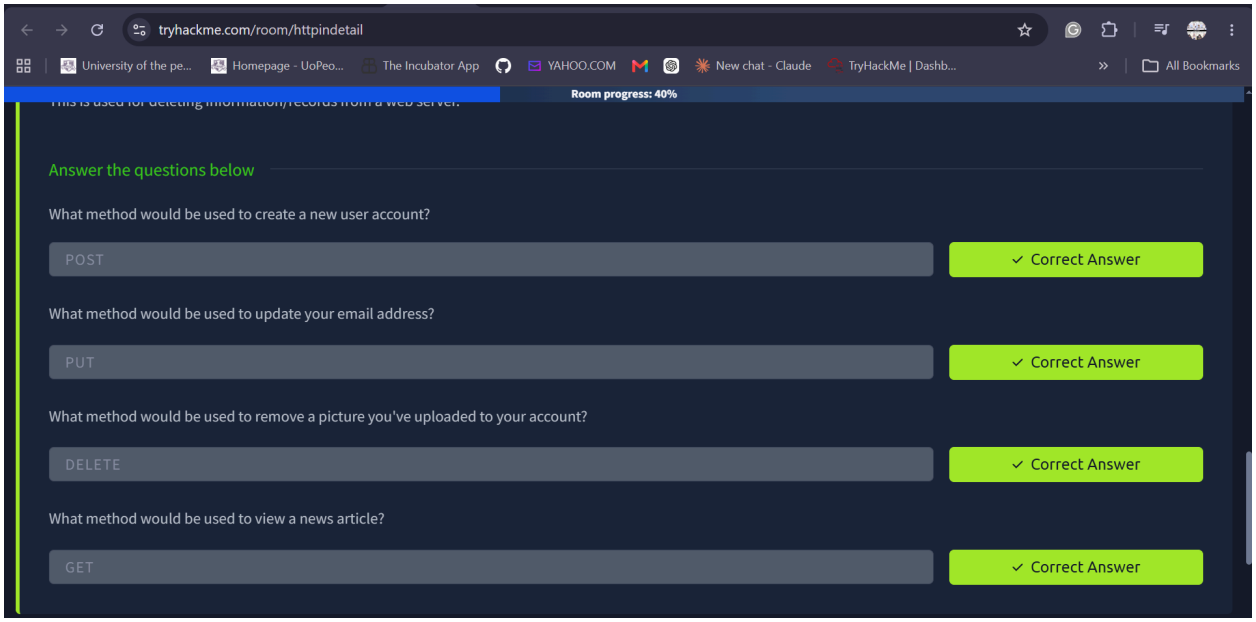

**Task 3: HTTP Methods**

**Notes:**

- **GET** retrieves information
- **POST** submits data and can create new records
- **PUT** submits data to update existing information
- **DELETE** removes information or records

GET and POST are the most common.

**Screenshot:**



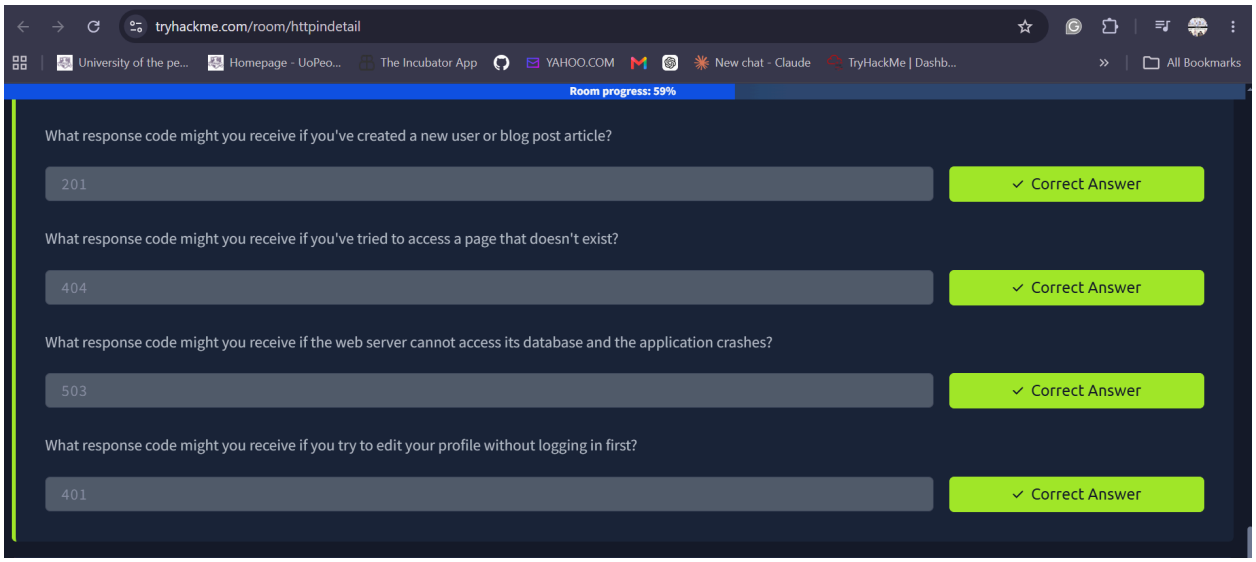

## Task 4: HTTP Status Codes

**Notes:**

Status codes are grouped into five ranges: **1xx** informational, **2xx** success, **3xx** redirection, **4xx** client errors and **5xx** server errors.

- **200** OK, **201** Created
- **301** Moved Permanently, **302** Found (temporary redirect)
- **400** Bad Request, **401** Not Authorised, **403** Forbidden, **404** Not Found, **405** Method Not Allowed
- **500** Internal Server Error, **503** Service Unavailable

**Screenshot:**



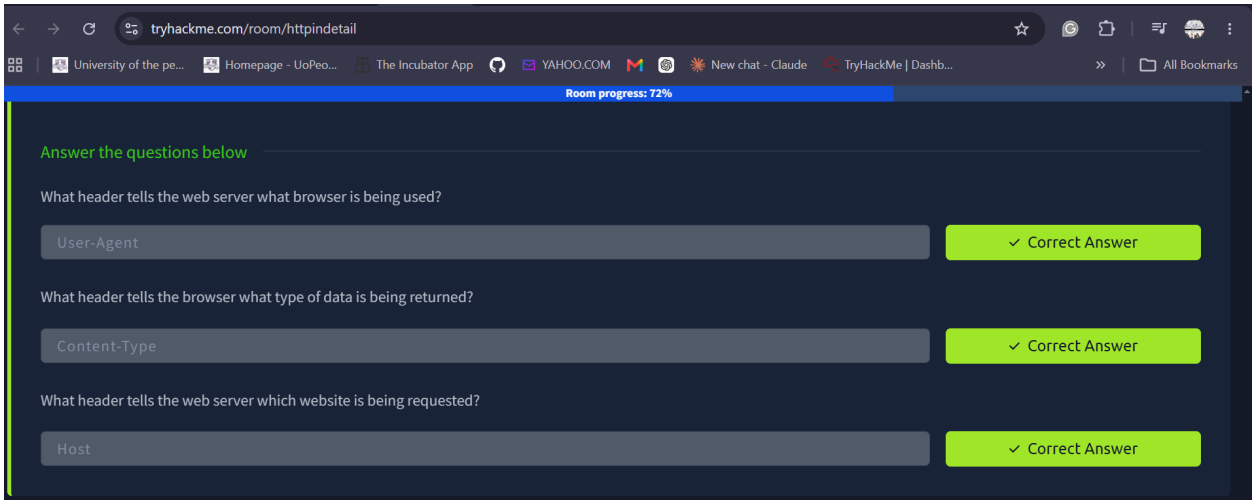

## Task 5: Headers

**Notes:**

Headers are extra pieces of data sent with requests and responses.

- **Request headers:** Host (which website I want), User-Agent (my browser and version), Content-Length (how much data is in the body), Accept-Encoding (compression methods my browser supports) and Cookie (data the server asked me to store).
- **Response headers:** Set-Cookie (data to store), Cache-Control (how long to keep the content cached), Content-Type (what kind of data is returned) and Content-Encoding (how it was compressed).

**Screenshot:**




**Task 6: Cookies**

**Notes:**

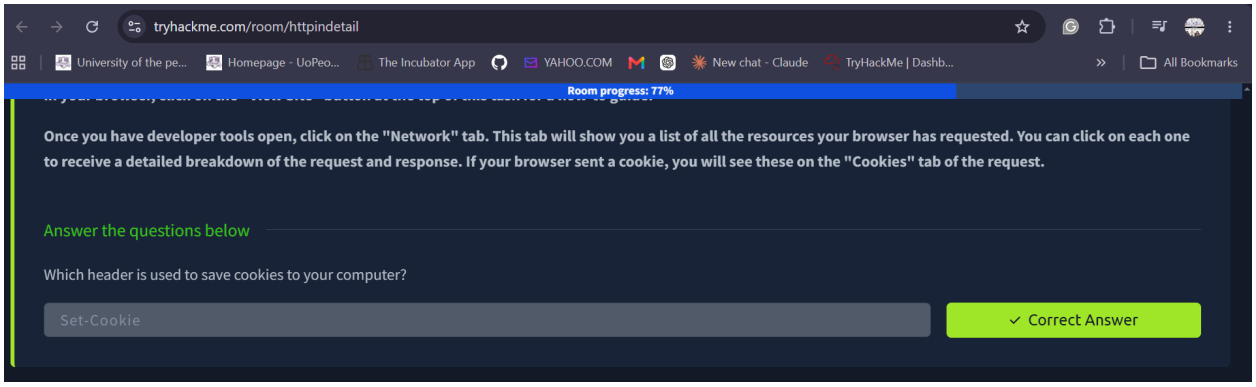
HTTP is **stateless**, meaning the server doesn't remember previous requests. Cookies solve this. The server sends a Set-Cookie header, the browser stores it, and then it sends the cookie back in a Cookie header on every later request. Cookies are most commonly used for **authentication**, and the value is usually a hard-to-guess token rather than a readable password.

The room's diagrams showed this flow:

1. The client sends GET / and the server replies with a page containing a form.
2. The client submits the form with a POST containing name=adam.
3. The server replies with Set-Cookie: name=adam.
4. On the next request, the client sends Cookie: name=adam, and the server shows "Welcome back adam" instead of the form.

I can view my cookies in the browser's developer tools under the **Network** tab.

**Screenshot:**




**Task 7: Hands-on Practice (HTTP Request Emulator)**

This is the practical part, so it's your strongest evidence. Include the request and response for each step.

| Step | Request | What it does | Result |
|---|---|---|---|
| 1 | GET /room | Retrieves the room page | 200 OK, flag returned |
| 2 | GET /blog with id=1 | Retrieves blog post 1 (query string parameter) | Flag returned |
| 3 | DELETE /user/1 | Deletes user 1 | Flag returned |
| 4 | PUT /user/2 with username=admin | Updates user 2 (body parameter) | Flag returned |
| 5 | POST /login with username=thm, password=letmein | Logs in (body parameters) | Flag returned |

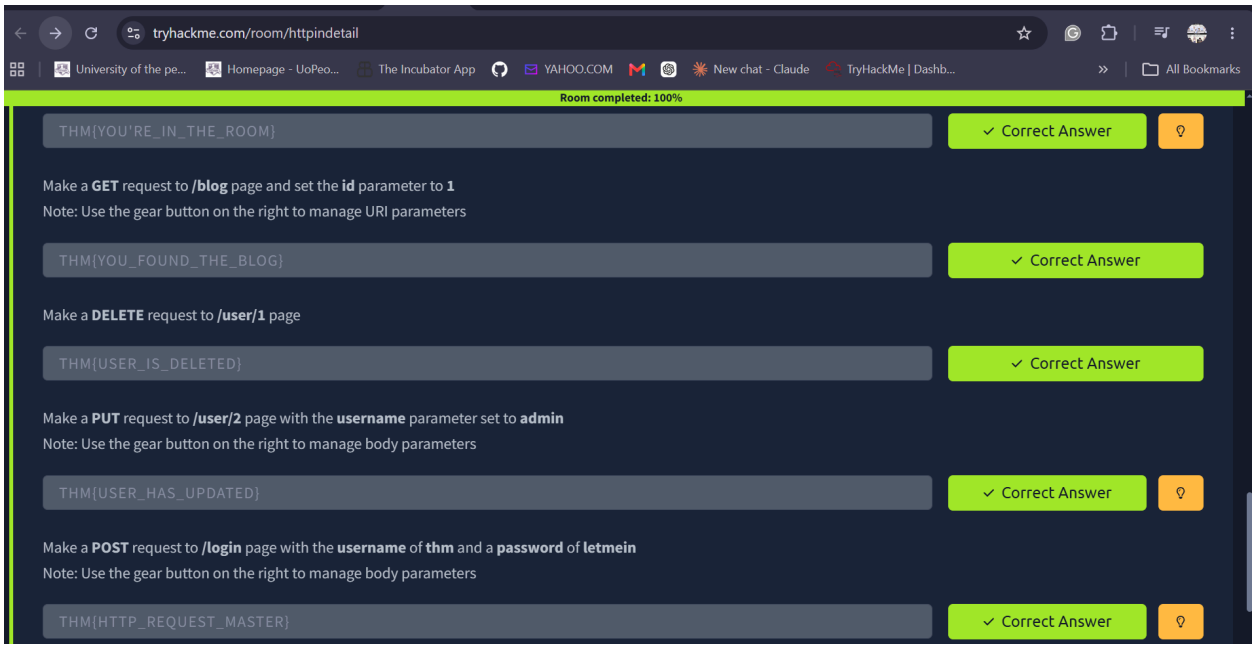
**Notes:**

I used the built-in HTTP emulator to send GET, DELETE, PUT and POST requests. Parameters can go in two places: in the **query string** (like id=1 on a GET request) or in the **body** (like the username and password on a POST or PUT request). I saw how the method and path together decide what the server does.

**Flags:**

- THM{YOU'RE_IN_THE_ROOM}
- THM{YOU_FOUND_THE_BLOG}
- THM{USER_IS_DELETED}
- THM{USER_HAS_UPDATED}
- THM{HTTP_REQUEST_MASTER}

**Screenshot:**




**Security observations to include**

- **Cookies are a target.** If someone steals or guesses a session cookie, they can act as that user without knowing the password.
- The DELETE and PUT requests changed data with **no login check** in the emulator. Real applications must verify the user is authorised first.
- The Server header (nginx/1.15.8) revealed the software and version, which helps an attacker look up known vulnerabilities.
- Credentials in a POST body are safer than in a URL, but only when HTTPS is used; otherwise, they travel in plain text.

**Final "What I learned" paragraph**

This room deepened my understanding of how HTTP works. I learned that HTTP is stateless and cookies are what let a server recognise me between requests. I practised sending GET, POST, PUT and DELETE requests and putting parameters in the query string and in the request body. I also learned to read status codes and headers, which will help me analyse traffic in Burp Suite.

**Completion link:**

https://tryhackme.com/room/httpindetail?utm_campaign=social_share&utm_medium=social&utm_content=share-completed-room&utm_source=copy&sharerId=6a298268d8b6b164348fab72